

Documentação de Arquitetura.

Veterano. — Plataforma de mentoria entre veteranos e calouros. Documentação parcial no padrão arc42, com C4 Model (níveis 1 a 4), UML, DER e Software Architecture Canvas.

GRUPO

Bento Barp

Guilherme Reis

Guilherme Selenko

Vinicius Trevisan

Repositório: github.com/ViniMTrevisan/pjbl-mentoria-cloud

Aplicação publicada: white-rock-03ee36c10.7.azurestaticapps.net · API: func-pjbl-mentoria.azurewebsites.net

Versão 1.0 · agosto de 2026 · documento parcial

Sumário

1. Introdução e Objetivos
2. Restrições da Arquitetura
3. Escopo e Contexto do Sistema — <i>C4 Nível 1</i>
4. Estratégia de Solução — <i>Software Architecture Canvas</i>
5. Visão de Blocos de Construção — <i>C4 Níveis 2, 3 e 4</i>
6. Visão de Runtime — <i>UML Sequência e Máquina de Estados</i>
7. Visão de Implantação — <i>UML Deployment</i>
8. Conceitos Transversais — <i>DER</i>
9. Decisões de Arquitetura
10. Requisitos de Qualidade
11. Riscos e Dívidas Técnicas
12. Glossário

Nota sobre o caráter parcial deste documento. A entrega cobre as doze seções do template arc42, mas o sistema descrito está implementado apenas em parte. Ao longo do texto, **IMPLEMENTADO** marca o que já está publicado em Azure e **PLANEJADO** marca o que está desenhado mas ainda não construído. A seção 5 e a seção 7 trazem o inventário completo desse recorte.

1. Introdução e Objetivos

Veterano. é uma plataforma web que conecta calouros que precisam de ajuda em uma matéria a veteranos dispostos a dar mentoria. Hoje esse encontro acontece de forma informal — grupos de WhatsApp, indicação pessoal, sorte — e depende de o calouro conhecer alguém. O sistema resolve o fluxo completo de ponta a ponta: **cadastro** → **descoberta** → **agendamento** → **avaliação**.

1.1 Visão geral dos requisitos

ID	REQUISITO FUNCIONAL	SITUAÇÃO
RF1	Cadastro e login do usuário.	PLANEJADO
RF2	Definir o perfil como mentor, mentorado ou ambos.	PLANEJADO
RF3	Mentor cadastra as matérias/áreas em que pode ajudar.	PLANEJADO
RF4	Mentor define sua disponibilidade de horários.	PLANEJADO
RF5	Mentorado busca mentores por curso, matéria ou área.	IMPLEMENTADO
RF6	Mentorado solicita uma sessão de mentoria.	PLANEJADO
RF7	Mentor aceita ou recusa a solicitação.	PLANEJADO
RF8	Notificar o usuário a cada mudança de status da solicitação.	PLANEJADO
RF9	Visualizar sessões futuras e histórico.	IMPLEMENTADO
RF10	Mentorado avalia o mentor após a sessão concluída.	PLANEJADO
RF11	Calcular e exibir a reputação do mentor.	IMPLEMENTADO
RF12	Marcar a sessão como concluída ou cancelada.	PLANEJADO

Ficam **fora de escopo** de forma deliberada, para manter o projeto viável no semestre: chat em tempo real, videochamada integrada, pagamento das mentorias, app mobile nativo e integração com o sistema acadêmico da instituição.

1.2 Metas de qualidade

#	ATRIBUTO	META CONCRETA
1	Usabilidade	O calouro conclui a busca por mentor no celular, sem zoom e sem instruções. A interface é toda em pt-BR, com datas em dd/MM/yyyy e fuso America/Sao_Paulo.
2	Segurança	Um usuário só lê e altera sessões em que é mentor ou mentorado. A autorização é verificada por recurso no servidor, nunca só na interface.
3	Confiabilidade	Uma falha no envio de e-mail não derruba a requisição. A notificação in-app é a fonte de verdade; o e-mail é <i>best effort</i> .
4	Custo	O custo mensal cabe no crédito da assinatura Azure for Students, com serviços serverless que escalam a zero quando ninguém usa.
5	Manutenibilidade	Toda regra de negócio vive no servidor e é testável sem interface. O front-end não recalcula reputação nem revalida transição de estado.

1.3 Stakeholders

PAPEL	QUEM É	O QUE ESPERA DA ARQUITETURA
Calouro (mentorado)	Aluno de 1º a 3º período	Achar rápido alguém que já passou pela matéria e agendar sem burocracia, pelo celular.
Veterano (mentor)	Aluno a partir do 4º período	Controlar sua disponibilidade, não ser incomodado fora dela e construir reputação visível.
Coordenação do curso	Professores e coordenadores	Moderar cadastros e enxergar métricas de uso sem acessar o banco diretamente.
Equipe de desenvolvimento	Bento, Guilherme R., Guilherme S., Vinicius	Poder trabalhar em paralelo em front-end e API, com contratos estáveis entre eles.
Professor da disciplina	Avaliador do PJBL	Rastrear cada decisão de arquitetura até um requisito e ver os princípios de cloud aplicados.

2. Restrições da Arquitetura

2.1 Restrições técnicas

RESTRIÇÃO	ORIGEM E CONSEQUÊNCIA
Hospedagem obrigatória em Microsoft Azure	Exigência da disciplina. Elimina alternativas como Vercel, Netlify ou AWS, e orienta toda a seleção de serviços da seção 4.
Front-end no Azure Static Web Apps	Exigência da atividade. Implica um artefato puramente estático: o front-end não pode ter renderização no servidor, o que descarta o Next.js com SSR previsto no PRD original.
Back-end em Azure Functions	Exigência da atividade. A API precisa ser stateless e tolerar <i>cold start</i> ; nenhum estado pode viver na memória do processo entre requisições.
Assinatura Azure for Students	Crédito limitado e sem cartão. Restringe a SKUs gratuitas ou de consumo e limita as regiões disponíveis (ver seção 7).
Regiões permitidas por policy	A policy <i>Allowed resource deployment regions</i> da assinatura libera apenas <code>centralus</code> , <code>mexicocentral</code> , <code>southafricanorth</code> , <code>canadacentral</code> e <code>chilecentral</code> . <code>brazilsouth</code> está bloqueada, apesar de ser a região mais próxima do usuário final.
Uso de IA generativa	Exigência da atividade. O código gerado precisa ser rastreável ao prompt que o originou (arquivo <code>Prompt.md</code>) e revisado por uma pessoa antes de publicar.

2.2 Restrições organizacionais e convenções

RESTRIÇÃO	CONSEQUÊNCIA
Equipe de quatro alunos, um semestre	O escopo foi cortado no MVP. Complexidade só entra na arquitetura quando paga um requisito explícito.
Sem orçamento para licenças	Todo o ferramental é gratuito ou open source: Vite, React, Node, PostgreSQL, GitHub.
Interface e documentação em pt-BR	Nomes de domínio no código em português (<code>Sessao</code> , <code>Disponibilidade</code>), reduzindo a tradução mental entre o requisito e o código.
Repositório público no GitHub	Nenhum segredo pode ser versionado. Credenciais ficam em variáveis de ambiente e, na evolução, no Azure Key Vault.

3. Escopo e Contexto do Sistema

Esta seção delimita a fronteira do sistema: quem interage com o Veterano. e quais sistemas externos ele precisa. O diagrama abaixo é o **Nível 1 do C4 Model** — o diagrama de contexto.

3.1 Contexto de negócio

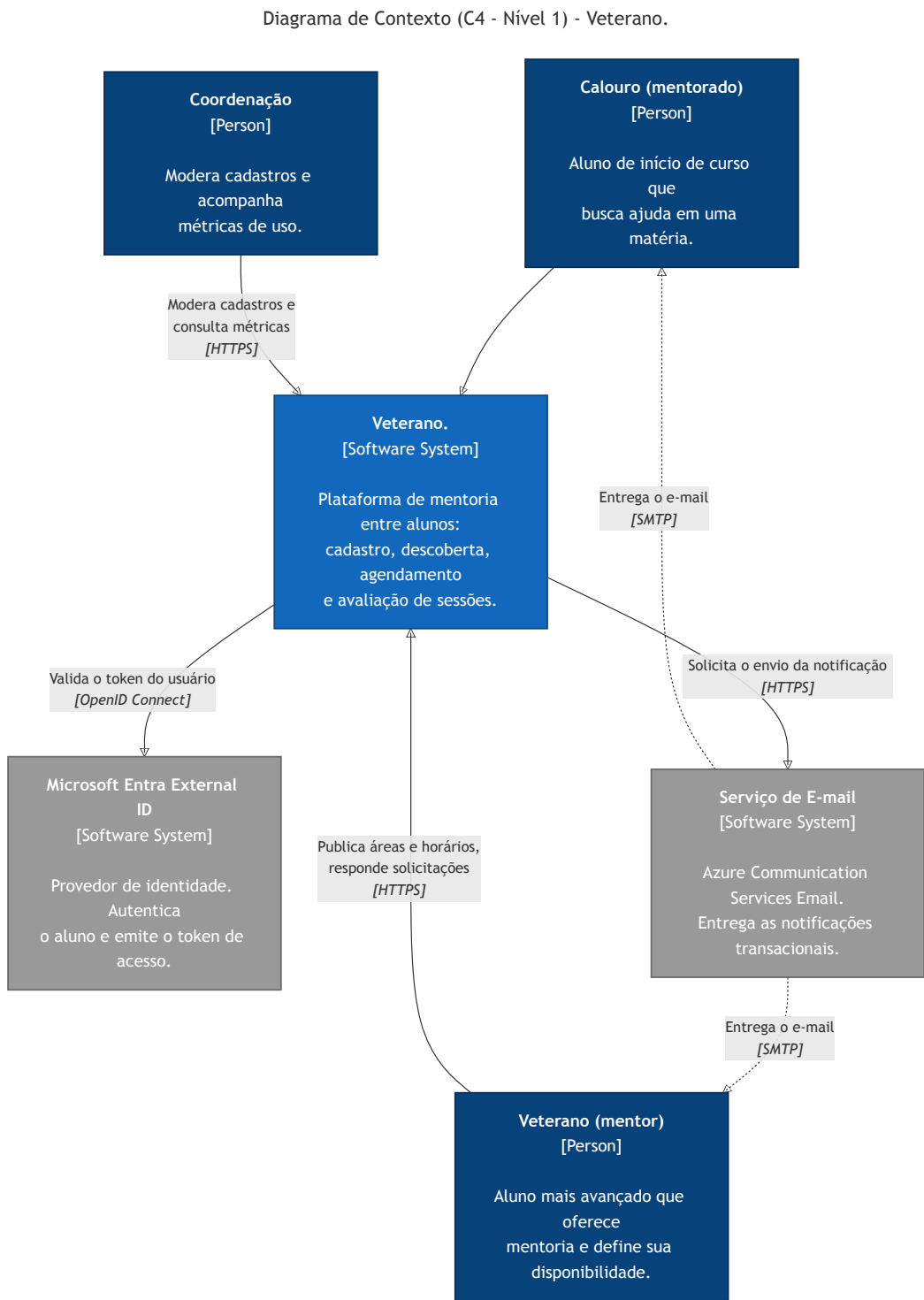


Figura 1. C4 Model — Nível 1 (Contexto). Fronteira do sistema, atores e sistemas externos.

PARCEIRO DE COMUNICAÇÃO	ENTRADA	SAÍDA
Calouro (mentorado)	Filtros de busca, solicitação de sessão, nota e comentário da avaliação.	Lista de mentores com reputação, situação das solicitações, notificações.
Veterano (mentor)	Áreas de mentoria, blocos de disponibilidade, decisão de aceitar ou recusar.	Solicitações recebidas, agenda de sessões, reputação acumulada.
Coordenação	Ações de moderação sobre cadastros.	Métricas gerais de uso da plataforma.
Microsoft Entra External ID	Credenciais do usuário no fluxo de login.	Token de identidade (JWT) validado pela API.
Serviço de e-mail	Destinatário, assunto e corpo da notificação.	Confirmação de aceite do envio; a entrega em si é assíncrona.

3.2 Contexto técnico

CANAL	PROTOCOLO	OBSERVAÇÃO
Navegador → Static Web Apps	HTTPS/TLS 1.2+	Entrega do bundle estático pela rede de borda do serviço. Certificado gerenciado pela Azure.
Navegador → Azure Functions	HTTPS, JSON	Chamada direta do browser à API. O CORS da Function App libera apenas a origem do Static Web App e <code>http://localhost:5173</code> para desenvolvimento.
SPA → Entra External ID	OpenID Connect, PKCE	O <i>authorization code flow</i> com PKCE é o adequado para cliente público sem segredo. PLANEJADO
Functions → PostgreSQL	TCP 5432 sobre TLS	Conexão autenticada por credencial vinda do Key Vault. PLANEJADO
Functions → Storage Queue	HTTPS	Enfileiramento das notificações por e-mail. PLANEJADO
Functions → Application Insights	HTTPS	Telemetria de requisições, dependências e exceções. IMPLEMENTADO

4. Estratégia de Solução

Um resumo das decisões estruturais que moldam a solução, e a justificativa de cada uma. As decisões estão detalhadas como ADRs na seção 9.

DECISÃO	ESCOLHA	POR QUÊ
Estilo arquitetural	SPA estática + API serverless (arquitetura <i>Jamstack</i> com back-end funcional)	O tráfego é intermitente e concentrado em época de prova. Serverless escala a zero fora desses picos e cabe no crédito de estudante.
Decomposição	Um contêiner por responsabilidade: SPA, API HTTP, processador de notificações, banco	Separa o que responde ao usuário (síncrono, sensível a latência) do que só precisa acontecer (envio de e-mail, assíncrono).
Persistência	PostgreSQL relacional, sem cache de reputação	O domínio é fortemente relacional e o volume é pequeno. Agregar as notas na leitura evita o problema de invalidação de cache sem custo perceptível.
Identidade	Provedor externo (Entra External ID), não autenticação própria	Guardar senha é risco que não paga benefício. Delegar tira o hash de senha, a recuperação e o bloqueio de conta do escopo da equipe.
Consistência do domínio	Máquina de estados explícita na Sessão, aplicada no servidor	É o núcleo do produto. Toda regra de quem pode fazer o quê fica em um único lugar testável.
Comunicação com o usuário	Notificação in-app como fonte de verdade; e-mail <i>best effort</i>	A entrega de e-mail depende de terceiros. Se falhar, o usuário ainda vê a notificação ao abrir o sistema.

4.1 Software Architecture Canvas

CONTEXTO E PROBLEMA - Calouros não têm canal estruturado para tirar dúvidas de matéria nem orientação de trajetória. - Veteranos dispostos a ajudar não têm onde se oferecer, controlar disponibilidade ou acumular reputação. - O encontro hoje depende de rede pessoal e sorte, e é invisível para a coordenação.	OBJETIVOS DE NEGÓCIO - Reduzir a evasão nos primeiros períodos. - Dar visibilidade ao veterano que ajuda. - Tornar a mentoria mensurável para a coordenação.	
USUÁRIOS E STAKEHOLDERS - Calouro (mentorado) — usuário primário. - Veterano (mentor) — oferta. - Coordenação — moderação e métricas. - Equipe de 4 alunos, um semestre.	REQUISITOS-CHAVE - RF5 descoberta com filtros. - RF6/RF7 solicitação e resposta. - RF9/RF12 agenda e ciclo de vida. - RF10/RF11 avaliação e reputação. - RF8 notificação a cada transição.	ATRIBUTOS DE QUALIDADE - Usabilidade no celular. - Autorização por recurso. - Degradação suave do e-mail. - Custo dentro do crédito. - Regra testável fora da UI.
RESTRIÇÕES - Azure obrigatório. - Static Web Apps + Functions. - Assinatura de estudante. - brazilsouth bloqueada por policy. - Repositório público — zero segredos.	PRINCÍPIOS - Regra de negócio no servidor. - Serviços gerenciados antes de infraestrutura própria. - Escalar a zero por padrão. - Complexidade só com requisito que a pague. - Estado terminal não volta atrás.	DECISÕES ESTRUTURAIS - SPA estática, sem SSR. - API HTTP stateless em Functions. - PostgreSQL relacional gerenciado. - Fila entre a API e o e-mail. - Identidade delegada ao Entra.
COMPONENTES - SPA React (3 telas). - API de Mentoria (HTTP triggers). - Núcleo de domínio (sessão, reputação). - Processador de notificações. - Repositórios de dados.	TECNOLOGIAS - React 19, Vite, TypeScript. - Azure Functions v4, Node 22. - Azure Static Web Apps (Free). - PostgreSQL Flexible Server. - Zod, Application Insights.	RISCOS - Latência de centralus ao Brasil. - Cold start na primeira chamada. - Crédito de estudante esgotar. - Mock ainda no lugar do banco. - Autenticação não implementada.
MÉTRICAS DE SUCESSO Produto: o roteiro completo do MVP (cadastrar → publicar disponibilidade → buscar → solicitar → aceitar → concluir → avaliar → ver reputação) executa de ponta a ponta na aplicação publicada. Arquitetura: busca de mentores responde em até 2 s no P95; falha de e-mail não gera erro HTTP; nenhum acesso a sessão de terceiro passa pela autorização. Operação: custo mensal dentro do crédito Azure for Students; toda exceção não tratada visível no Application Insights.		

5. Visão de Blocos de Construção

A decomposição estática do sistema, refinada em três níveis. O Nível 1 corresponde ao **C4 Nível 2 (contêineres)**; o Nível 2, ao **C4 Nível 3 (componentes)**; e o Nível 3, ao **C4 Nível 4 (código)**, detalhado em diagramas UML de classes e de componentes.

5.1 Nível 1 — visão caixa-branca do sistema (C4 Nível 2: Contêineres)

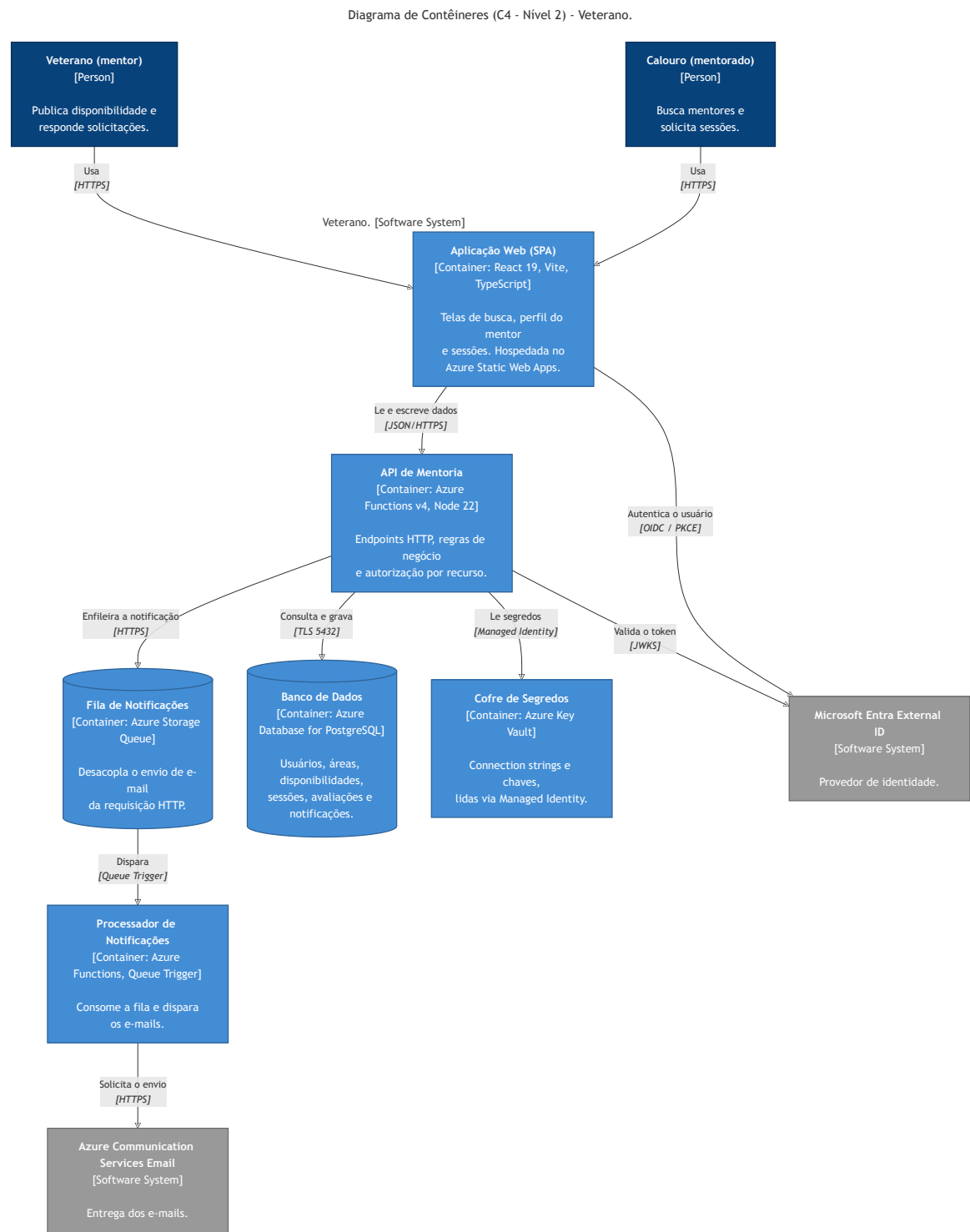


Figura 2. C4 Model — Nível 2 (Contêineres). Unidades executáveis e implantáveis do Veterano. e como se comunicam.

CONTÊINER	TECNOLOGIA	RESPONSABILIDADE	SITUAÇÃO
Aplicação Web (SPA)	React 19, Vite, TypeScript	Renderiza as telas, coleta a entrada do usuário e formata datas e textos em pt-BR. Não contém regra de negócio.	IMPLEMENTADO
API de Mentoria	Azure Functions v4, Node 22	Expõe os endpoints HTTP, valida o payload, aplica a autorização por recurso e as regras de domínio.	IMPLEMENTADO (somente leitura, dados mock)
Processador de Notificações	Azure Functions, Queue Trigger	Consome a fila e chama o serviço de e-mail. Isolado para que uma falha de entrega não afete a API.	PLANEJADO
Fila de Notificações	Azure Storage Queue	Desacopla o envio de e-mail do ciclo da requisição HTTP.	PLANEJADO
Banco de Dados	Azure Database for PostgreSQL Flexible Server	Guarda todo o estado do sistema. Hoje substituído por um módulo de dados mock em memória.	PLANEJADO
Cofre de Segredos	Azure Key Vault	Guarda connection strings e chaves, lidas pela API via Managed Identity.	PLANEJADO

5.2 Nível 2 — dentro da API de Mentoria (C4 Nível 3: Componentes)

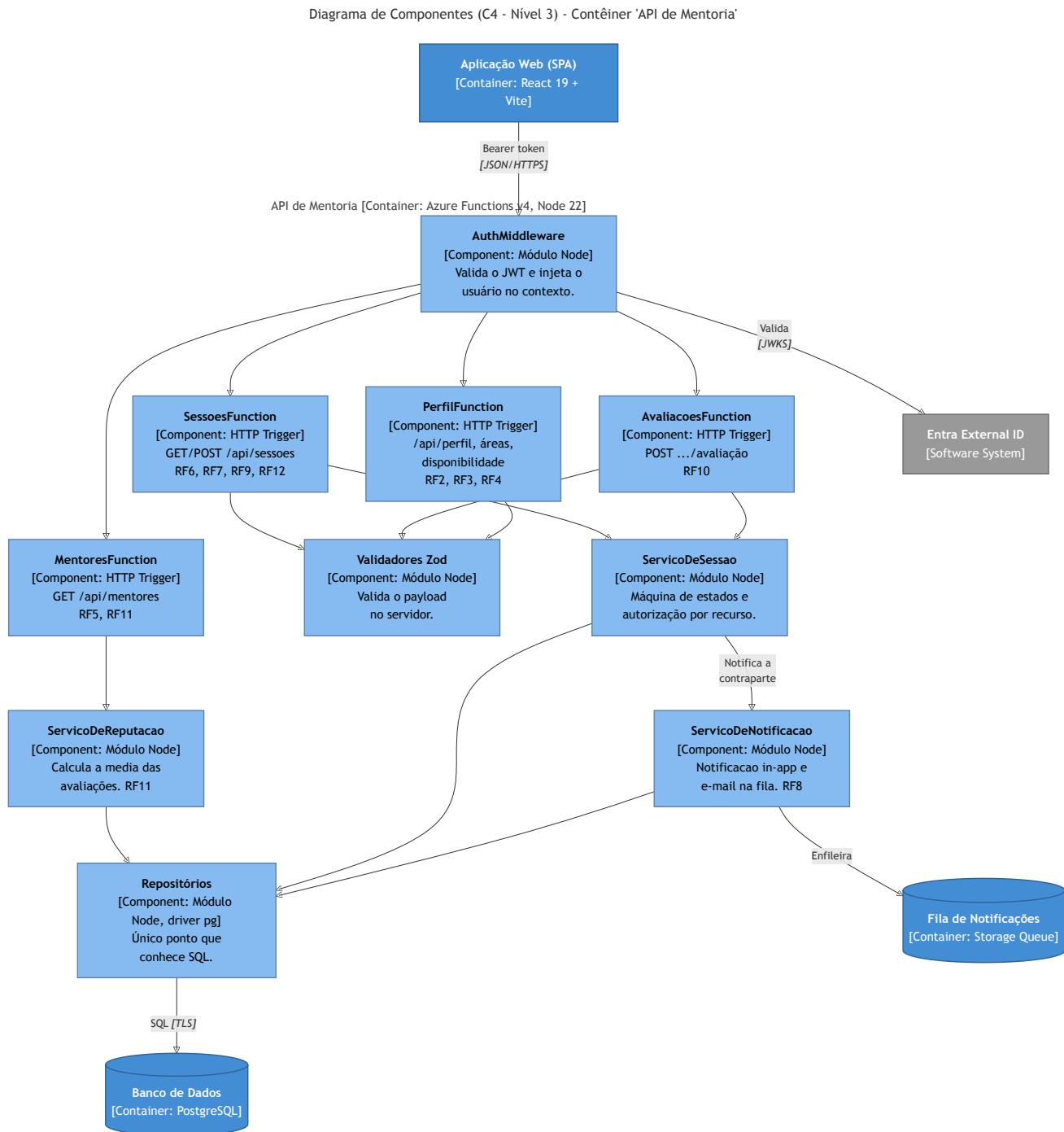


Figura 3. C4 Model — Nível 3 (Componentes). Decomposição interna do contêiner "API de Mentoria".

COMPONENTE	RESPONSABILIDADE	RF
AuthMiddleware	Valida a assinatura do JWT contra o JWKS do Entra e injeta o usuário autenticado no contexto. Toda rota exceto cadastro e login passa por aqui.	RF1
MentoresFunction	Descoberta de mentores e perfil público. Filtra por curso, matéria e palavra-chave e ordena por reputação decrescente.	RF5, RF11
SessoesFunction	Cria a solicitação, lista as sessões do usuário e recebe as ações de transição de estado.	RF6, RF7, RF9, RF12
AvaliacoesFunction	Registra a avaliação, aceitando apenas do mentorado e apenas uma vez por sessão concluída.	RF10
PerfilFunction	Perfil e papéis do usuário, áreas de mentoria e blocos de disponibilidade.	RF2, RF3, RF4
ServiçoDeSessão	Guarda a máquina de estados e a autorização por recurso. Nenhuma transição acontece fora dele.	RF6, RF7, RF12
ServiçoDeReputação	Agrega as notas na leitura e devolve média com uma casa decimal e total, ou a ausência explícita de avaliações.	RF11
ServiçoDeNotificação	Grava a notificação in-app e enfileira o e-mail a cada transição de estado.	RF8
Validadores Zod	Validam o corpo da requisição no servidor antes de qualquer escrita. Os mesmos schemas alimentam o formulário no front-end.	todos
Repositórios	Encapsulam o acesso ao PostgreSQL. É o único ponto do sistema que conhece SQL.	todos

5.3 Nível 3 — código (C4 Nível 4)

5.3.1 Diagrama de classes (UML)

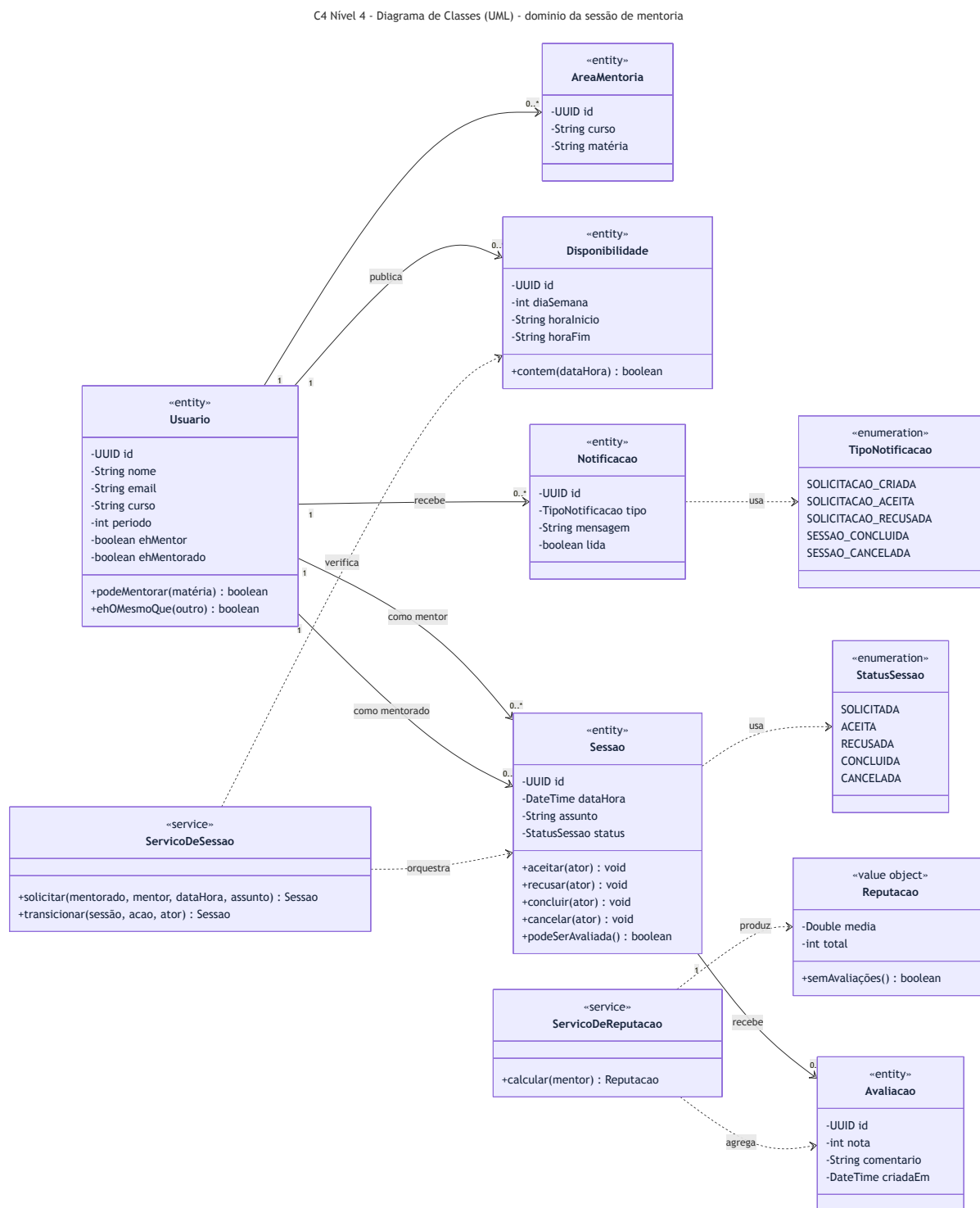


Figura 4. C4 Model — Nível 4 (Código). Diagrama de classes UML do domínio da sessão de mentoria.

O modelo segue a regra do PRD de **não separar mentor e mentorado em entidades distintas**: há uma única classe **Usuario**, com os papéis representados pelos atributos **ehMentor** e **ehMentorado**, e a **Sessao** se liga a ela por dois relacionamentos de papéis diferentes. A **Reputacao** é um *value object* calculado na leitura, sem coluna persistida — o que elimina a necessidade de invalidação quando uma avaliação nova chega.

5.3.2 Diagrama de componentes (UML)

C4 Nível 4 - Diagrama de Componentes (UML) - módulos internos da API

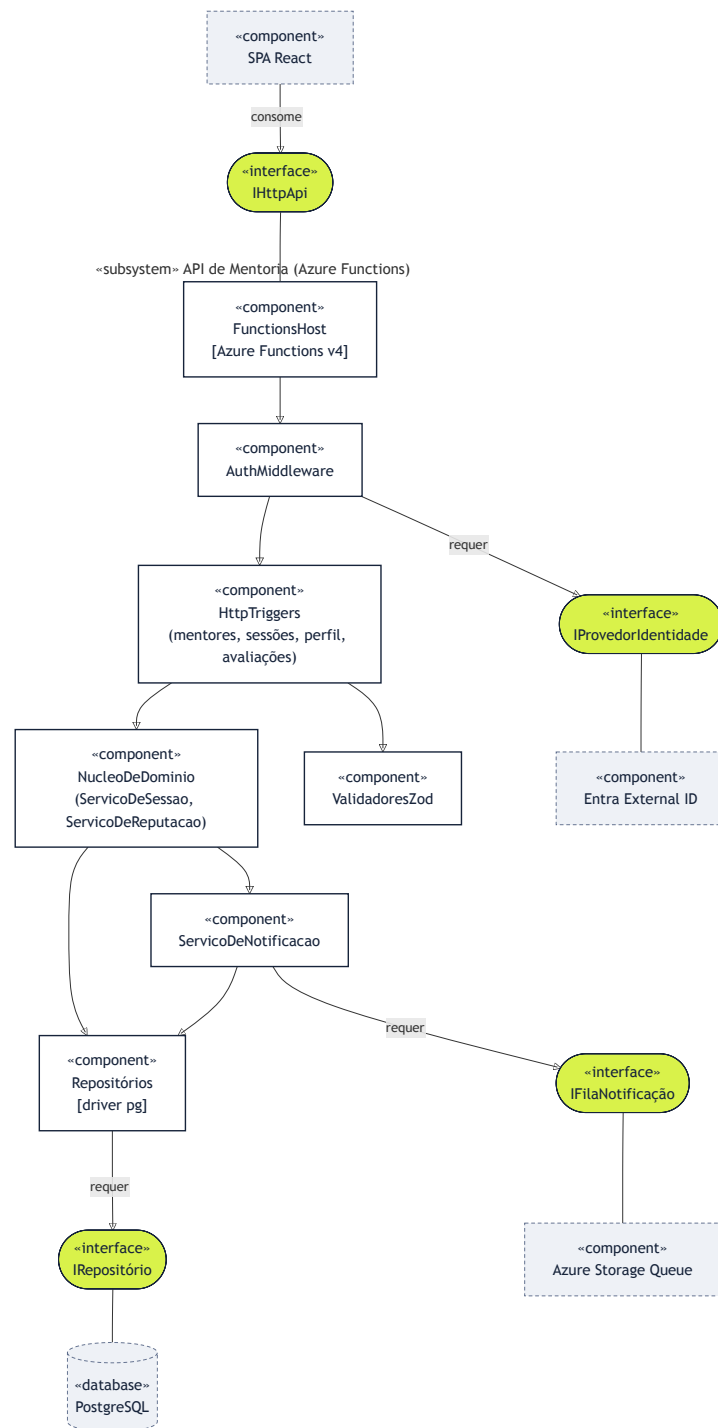


Figura 5. C4 Model — Nível 4 (Código). Diagrama de componentes UML, com interfaces providas e requeridas.

O diagrama usa a notação UML de componentes com interfaces providas e requeridas. A dependência aponta sempre para dentro: os *triggers* HTTP dependem do núcleo de domínio, e o núcleo depende de `IRepositorio` e `IFilaNotificacao` como abstrações — não das implementações concretas do PostgreSQL ou da Storage Queue. É essa inversão que permite trocar o mock atual pelo banco real sem tocar nas regras de negócio.

6. Visão de Runtime

Como os blocos de construção colaboram em tempo de execução. O cenário escolhido é o caminho crítico do produto — solicitar e responder uma sessão — porque é onde vivem a autorização, a máquina de estados e a notificação.

6.1 Cenário: solicitar e aceitar uma sessão (RF6 e RF7)

Três aspectos do cenário merecem destaque, e aparecem no diagrama a seguir:

- **A validação acontece duas vezes, em camadas diferentes.** O `ValidadorZod` rejeita o payload malformatado (400); o `ServicoDeSessao` rejeita o payload bem formado mas inválido no domínio — horário fora da disponibilidade ou no passado (422).
- **A autorização é do ator, não da rota.** Só o mentor daquela sessão específica pode aceitar. Um mentorado tentando aceitar a própria solicitação recebe 403, mesmo estando autenticado.
- **O e-mail sai do caminho síncrono.** A API enfileira e responde. Se a fila ou o serviço de e-mail estiver fora do ar, a sessão já foi criada e a notificação in-app já existe.

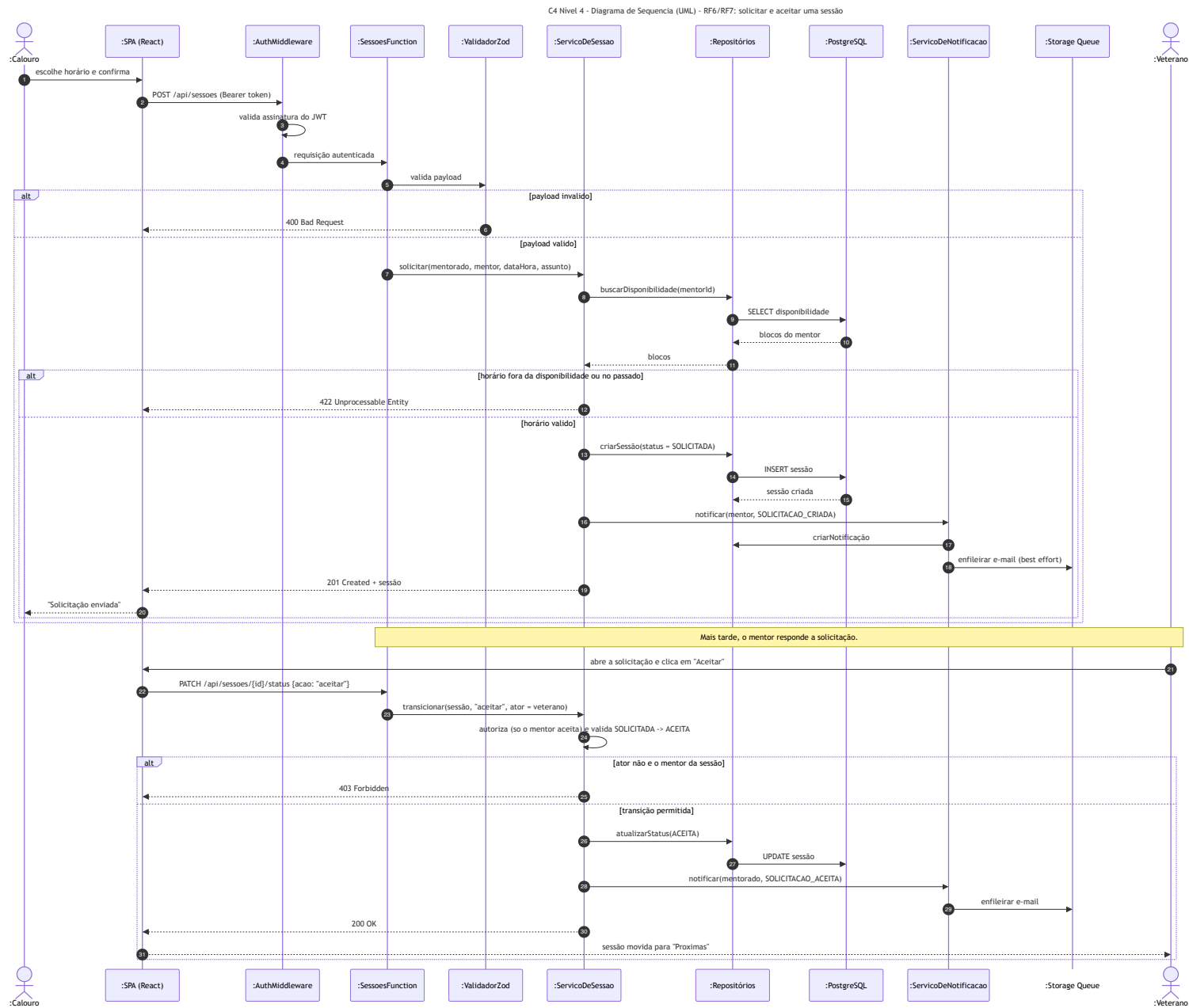


Figura 6. C4 Model — Nível 4 (Código). Diagrama de sequência UML: solicitação e aceite de uma sessão (RF6 e RF7).

6.2 Ciclo de vida da Sessão

As regras de transição estão resumidas abaixo e representadas na máquina de estados da figura seguinte.

AÇÃO	QUEM PODE	A PARTIR DE	REGRA
Solicitar	Mentorado	—	O horário precisa estar dentro da disponibilidade declarada do mentor e no futuro. Não é permitido solicitar sessão consigo mesmo.
Aceitar	Apenas o mentor da sessão	SOLICITADA	Move para ACEITA e notifica o mentorado.
Recusar	Apenas o mentor da sessão	SOLICITADA	Estado terminal.
Cancelar	Mentor ou mentorado	SOLICITADA ou ACEITA	Estado terminal, disponível para os dois lados.
Concluir	Mentor ou mentorado	ACEITA	Libera a avaliação.
Avaliar	Apenas o mentorado	CONCLUIDA	Uma única vez por sessão, com nota de 1 a 5 e comentário opcional.

Máquina de estados da Sessão (UML Staté Machine)

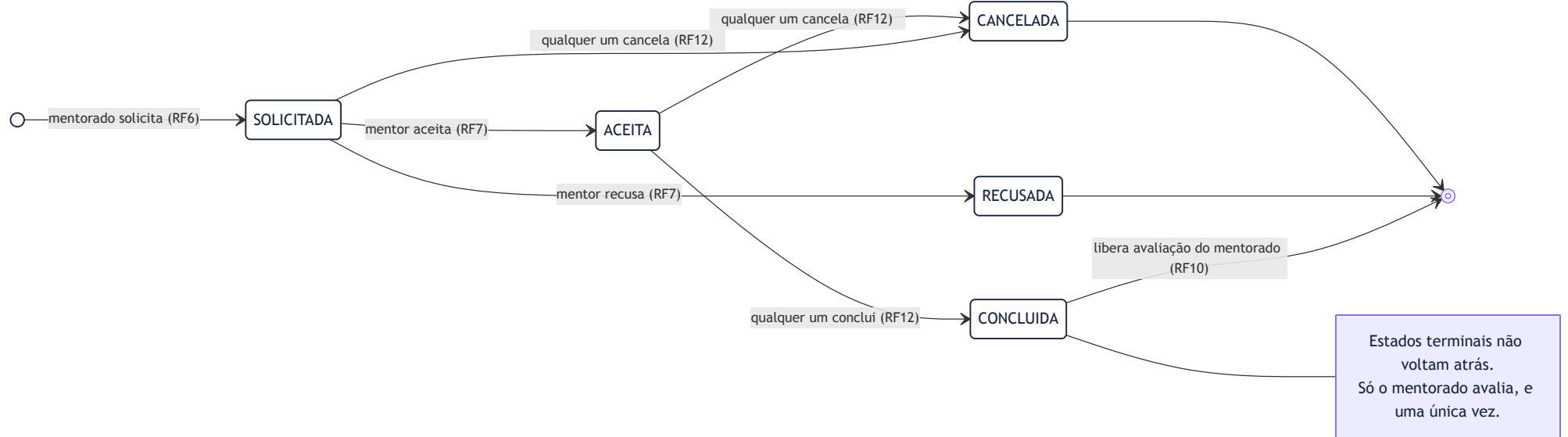


Figura 7. Diagrama de máquina de estados UML da entidade Sessão.

7. Visão de Implantação

A infraestrutura de execução e o mapeamento dos blocos de construção sobre ela.

7.1 Recursos provisionados

RECURSO	NOME	SKU / PLANO	PAPEL
Resource Group	rg-pjbl-mentoria	—	Agrupar todo o ciclo de vida do projeto. Apagar o grupo apaga a solução inteira.
Static Web App	swa-pjbl-mentoria-7205	Free	Serve o bundle do Vite pela rede de borda, com HTTPS e certificado gerenciados.
Function App	func-pjbl-mentoria	Flex Consumption, Node 22	Executa a API HTTP. Escala a zero quando não há requisição.
Storage Account	stpjbldmentoria7205	Standard_LRS	Guarda o pacote de deploy da Function App e, na evolução, a fila de notificações.
Application Insights	func-pjbl-mentoria	—	Telemetria de requisições, dependências e exceções.
PostgreSQL Flexible Server	—	Burstable B1ms	Persistência. PLANEJADO
Key Vault	—	Standard	Segredos acessados por Managed Identity. PLANEJADO

Diagrama de Implantação (UML Deployment) - Azure

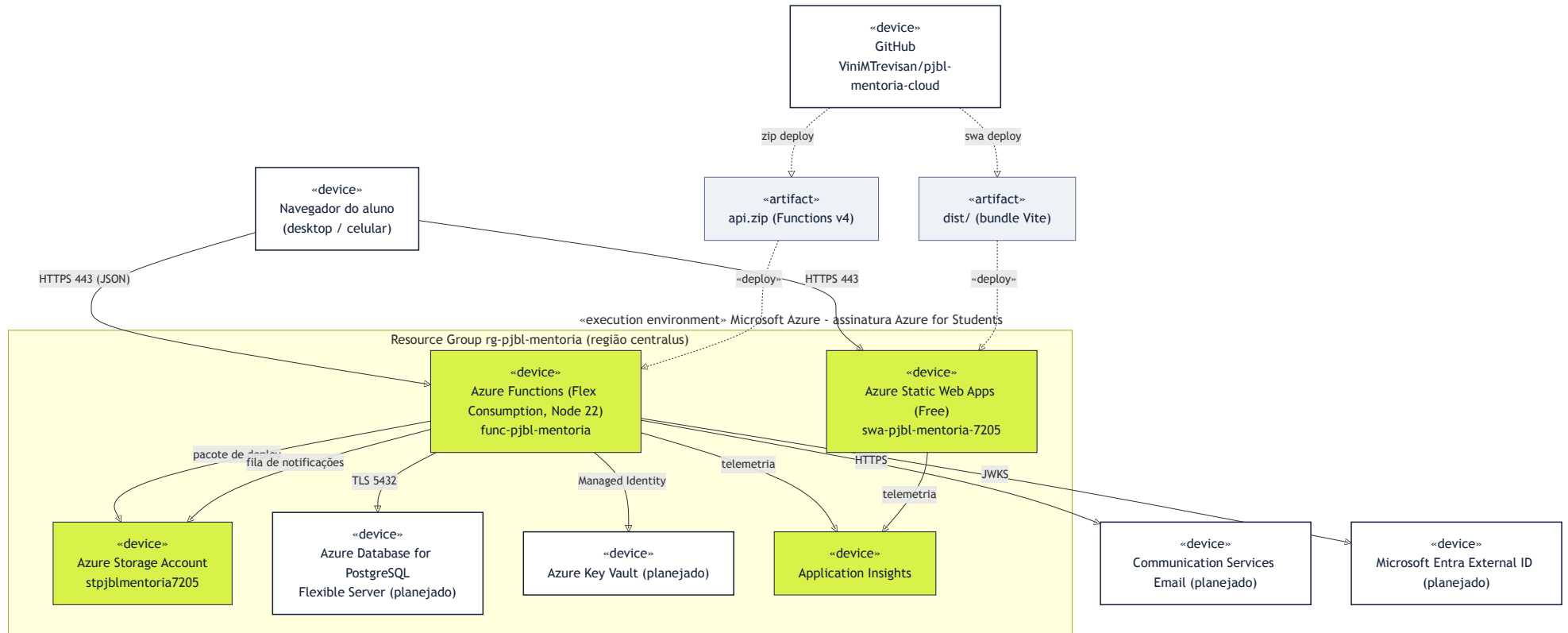


Figura 8. Diagrama de implantação UML: nós de execução em Azure e artefatos publicados.

7.2 Escolha de região

Todos os recursos estão em `centralus`. Essa não foi a primeira escolha: `brazilsouth` seria a região mais próxima do usuário final, mas a policy *Allowed resource deployment regions* da assinatura Azure for Students a bloqueia. Entre as cinco regiões liberadas, `centralus` é a única que também suporta Azure Static Web Apps, o que permitiu manter o front-end e a API na mesma região e evitar tráfego entre regiões. O custo dessa restrição é uma latência maior para o usuário no Brasil, registrada como risco na seção 11.

7.3 Pipeline de publicação

ARTEFATO	ORIGEM	COMO É PUBLICADO
Bundle do front-end	<code>frontend/dist</code> (build do Vite)	<code>swa deploy</code> com <i>deployment token</i> do Static Web App. A URL da API entra no build pela variável <code>VITE_API_BASE_URL</code> .
Pacote da API	<code>api.zip</code>	<code>az functionapp deployment source config-zip</code> , com <i>sync triggers</i> automático ao final.
Infraestrutura	<code>infra.sh</code>	Script idempotente em Azure CLI: cria resource group, storage, Function App e Static Web App.

8. Conceitos Transversais

8.1 Modelo de domínio e persistência — Diagrama de Entidades e Relacionamentos

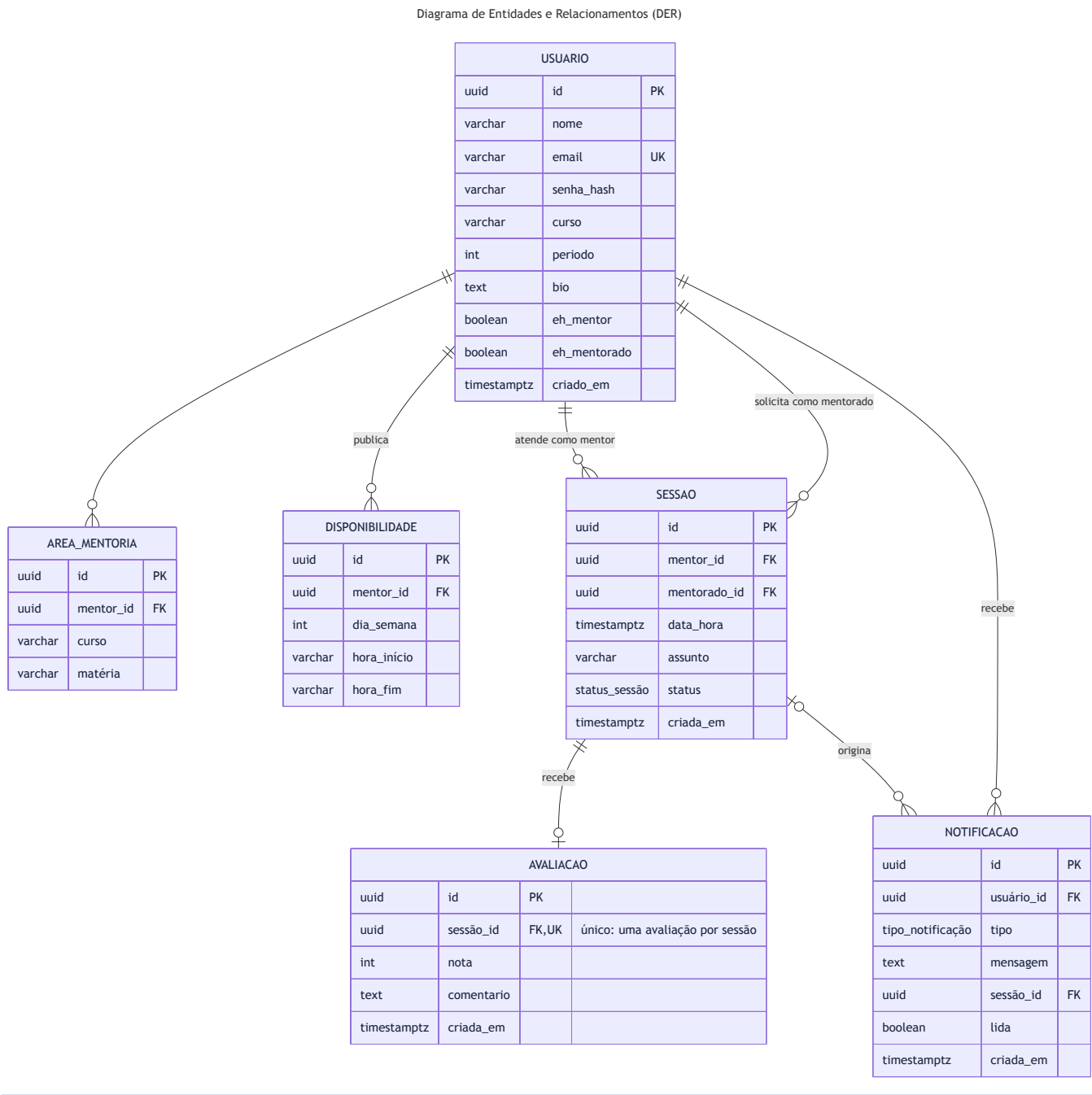


Figura 9. Diagrama de Entidades e Relacionamentos (DER) do modelo persistido.

DECISÃO DE MODELAGEM	JUSTIFICATIVA
Uma única tabela <code>USUARIO</code> para mentor e mentorado	Um mesmo aluno pode ser as duas coisas ao mesmo tempo. Duas tabelas exigiriam duplicar o cadastro e sincronizar dois registros da mesma pessoa.
Dois relacionamentos distintos entre <code>USUARIO</code> e <code>SESSAO</code>	<code>mentor_id</code> e <code>mentorado_id</code> apontam para a mesma tabela com papéis diferentes. A restrição de que os dois não podem ser iguais é aplicada no domínio.
<code>AVALIACAO</code> com <code>sessao_id</code> único	Impõe no banco a regra de uma avaliação por sessão, em vez de confiar apenas na verificação da aplicação.
Reputação sem coluna persistida	Calculada por agregação na leitura. Elimina invalidação de cache; o volume de avaliações por mentor é pequeno o bastante para tornar o custo irrelevante.
<code>NOTIFICACAO</code> com <code>sessao_id</code> opcional	Nem toda notificação nasce de uma sessão; o campo é anulável para acomodar avisos de moderação no futuro.
Chaves UUID	Não expõem volume nem ordem de cadastro em URLs públicas, ao contrário de identificadores sequenciais.

8.2 Segurança

- **Autenticação delegada.** A identidade fica no Entra External ID. A API valida a assinatura do token contra o JWKS do provedor e nunca guarda senha. Enquanto o provedor não está integrado, os endpoints publicados são de leitura sobre dados fictícios e ficam com `authLevel: anonymous`.
- **Autorização por recurso.** Estar autenticado não basta: cada operação verifica se o usuário é o mentor ou o mentorado *daquela* sessão. Tentativa de acesso a sessão de terceiro responde 403.
- **Segredos fora do repositório.** O repositório é público. Connection strings e chaves ficam em configuração de aplicação e, na evolução, no Key Vault acessado por Managed Identity — sem credencial no código.
- **CORS restritivo.** A Function App aceita apenas a origem do Static Web App e `http://localhost:5173`, em vez do curinga `*`.
- **Senha (caso a autenticação própria seja retomada).** Hash `bcrypt`, nunca em texto plano e nunca em log.

8.3 Validação e tratamento de erros

- Os mesmos schemas Zod validam no formulário e no servidor. A validação no cliente é conveniência; a do servidor é a que vale.
- Códigos de resposta com significado consistente: `400` payload malformatado, `403` ator sem permissão sobre o recurso, `404` recurso inexistente, `422` payload válido mas proibido pela regra de negócio.
- Toda tela trata três situações além do caminho feliz: carregando, vazio e erro. Nenhuma falha de rede resulta em tela branca — a interface explica o que houve, na voz do sistema.

8.4 Internacionalização e formatação

Interface inteiramente em pt-BR. Datas em `dd/MM/yyyy` e horários no fuso `America/Sao_Paulo`, formatados com `Intl.DateTimeFormat` no cliente a partir de timestamps com fuso vindos da API — o servidor nunca envia data já formatada.

8.5 Observabilidade

Application Insights coleta requisições, dependências e exceções da Function App. Cada handler registra a operação e os filtros recebidos, sem incluir dado pessoal no log. Falhas de envio de e-mail são registradas como aviso, não como erro da requisição — coerente com o tratamento *best effort* descrito na seção 4.

8.6 Experiência e interface

O front-end é responsivo com abordagem *mobile first*, porque o calouro abre o sistema no celular entre uma aula e outra. A identidade visual parte do mundo do aluno — papel quadriculado e marca-texto — e reserva a cor de destaque para uma única informação: o que está disponível. O elemento de maior peso visual é a grade semanal de disponibilidade do mentor, desenhada como a grade horária da faculdade. Foco de teclado visível e `prefers-reduced-motion` respeitado.

8.7 Uso de IA generativa

O front-end, as Azure Functions e os scripts de infraestrutura foram gerados com assistência do Claude Code, com acesso ao Azure MCP Server e à Azure CLI. Os prompts estão versionados em `Prompt.md`, no repositório. Todo código gerado passou por revisão humana antes da publicação: conferência das regras de negócio contra o PRD, teste dos endpoints e navegação nas três telas no ambiente publicado.

9. Decisões de Arquitetura

Decisões registradas no formato ADR. Só entram aqui as que foram custosas de tomar ou que seriam caras de reverter.

ADR-01 — Substituir o Next.js full-stack por SPA estática mais API serverless

CONTEXTO

O PRD original previa Next.js com App Router, Route Handlers e SSR em um único projeto, publicado na Vercel. A disciplina exige Azure, e a atividade exige Azure Static Web Apps com Azure Functions.

DECISÃO

Separar em dois artefatos: uma SPA React + Vite puramente estática e uma API em Azure Functions.

CONSEQUÊNCIAS

Positivas: o front-end vira arquivo estático, servido pela borda a custo próximo de zero; front-end e API escalam e são publicados de forma independente; o contrato entre eles fica explícito. **Negativas:** perdemos SSR e Server Actions, o que piora o SEO (irrelevante aqui, já que o conteúdo é autenticado) e obriga a tratar estados de carregamento manualmente em cada tela.

ALTERNATIVAS DESCARTADAS

Next.js com adaptador para Static Web Apps (SSR híbrido) — mais partes móveis do que a equipe conseguiria operar no semestre.

ADR-02 — Flex Consumption em vez de Consumption clássico

CONTEXTO

A Function App criada no plano Consumption em Linux respondia HTTP 503 em todos os endpoints, inclusive no site de deploy (SCM), sem que o host chegasse a iniciar. O plano também tem fim de suporte anunciado.

DECISÃO

Recriar a Function App em Flex Consumption com Node 22.

CONSEQUÊNCIAS

Positivas: a aplicação subiu na primeira tentativa; *cold start* menor e plano com suporte de longo prazo. **Negativas:** o comando `az functionapp deploy` não funciona nesse plano (responde 415) — a publicação usa `az functionapp deployment source config-zip`, o que precisa estar documentado para quem for publicar depois.

ADR-03 — Function App separada, e não as managed functions do Static Web App

CONTEXTO

O Static Web Apps permite hospedar funções na pasta `/api` do próprio site, o que dispensaria configurar CORS e um segundo recurso.

DECISÃO

Manter a API como Function App independente.

CONSEQUÊNCIAS

Positivas: a API tem URL pública própria e pode ser consumida por outros clientes; escala e é publicada separadamente do front-end; a entrega da disciplina exige o endereço da Azure Function. **Negativas:** exige configurar CORS explicitamente e gerenciar mais um recurso.

ADR-04 — Reputação calculada na leitura, sem coluna materializada

CONTEXTO

A reputação (RF11) aparece na busca e no perfil, e muda a cada avaliação nova. Guardá-la em coluna evitaria a agregação.

DECISÃO

Calcular por agregação na leitura, sem persistir.

CONSEQUÊNCIAS

Positivas: nenhuma invalidação de cache, nenhum risco de divergência entre a média exibida e as notas reais. **Negativas:** a consulta de busca agrega em toda leitura — aceitável no volume esperado, e revisável se a listagem crescer.

ADR-05 — Notificação in-app como fonte de verdade, e-mail assíncrono

CONTEXTO

O RF8 pede notificação a cada mudança de status. A entrega de e-mail depende de terceiro e pode falhar ou demorar.

DECISÃO

Gravar a notificação in-app dentro da transação da transição e enfileirar o e-mail para um processador separado.

CONSEQUÊNCIAS

Positivas: a requisição HTTP não fica presa a um serviço externo; falha de e-mail não vira erro para o usuário; retentativa fica isolada no consumidor da fila. **Negativas:** mais um contêiner e uma fila para operar, e o e-mail passa a ter latência variável.

ADR-06 — Dados mock dentro das Functions, sem serviço externo de mock

CONTEXTO

A atividade sugeria o Apidog como plataforma de mock. O banco ainda não existe e a API precisa responder dados coerentes.

DECISÃO

Manter os dados fictícios em um módulo dentro da própria Function App, atrás dos mesmos handlers que atenderão o banco real.

CONSEQUÊNCIAS

Positivas: as regras de negócio (filtro, ordenação, ausência de avaliações) já rodam no servidor e continuam valendo quando o banco entrar; nenhuma dependência de conta em serviço de terceiro; o contrato da API é o definitivo. **Negativas:** trocar o mock pelo banco exige alterar código, não apenas apontar para outra URL.

10. Requisitos de Qualidade

10.1 Árvore de qualidade

Árvore de Requisitos de Qualidade (arc42 seção 10)

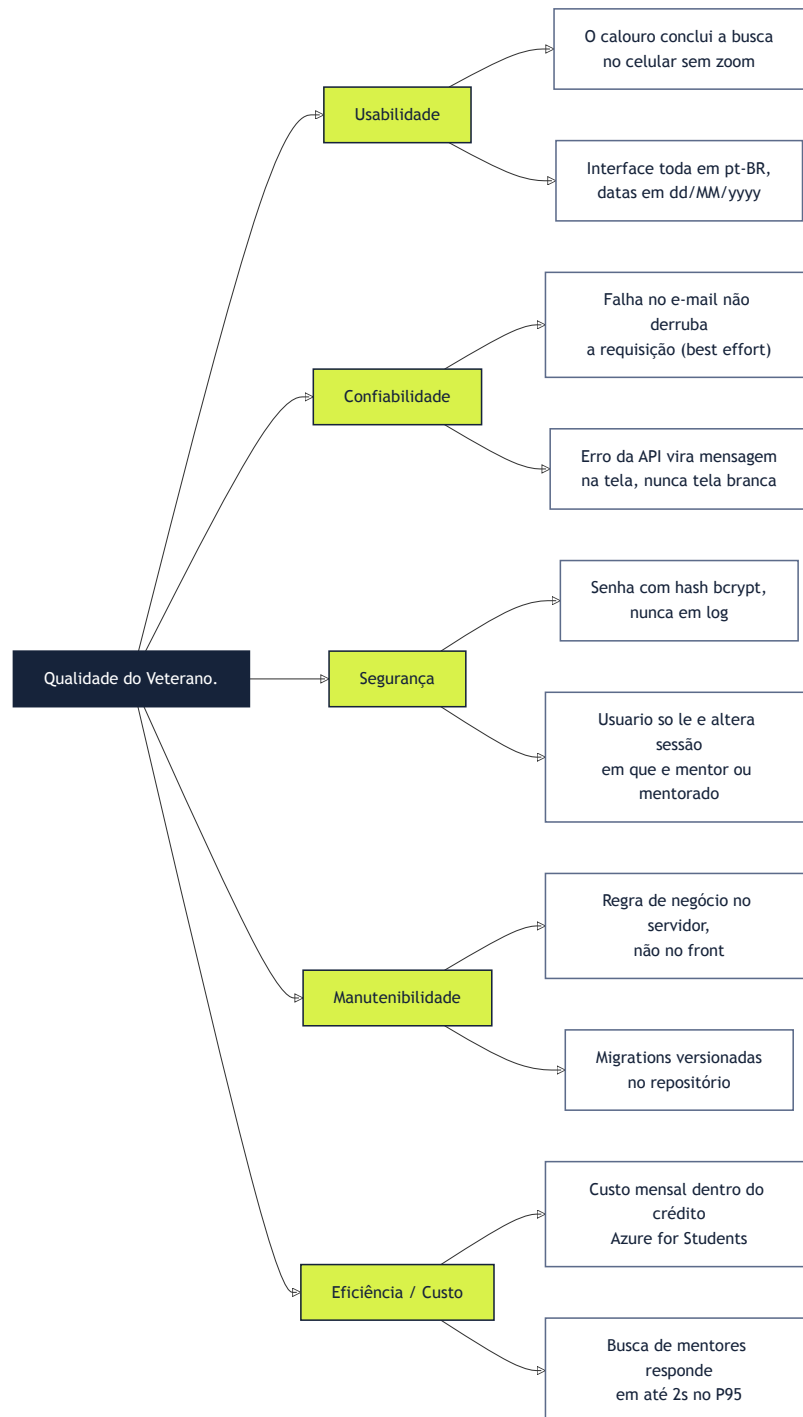


Figura 10. Árvore de requisitos de qualidade (arc42, seção 10).

10.2 Cenários de qualidade

#	ATRIBUTO	ESTÍMULO	RESPOSTA ESPERADA
Q1	Usabilidade	Um calouro abre a plataforma no celular pela primeira vez, sem treinamento.	Encontra e filtra mentores pela matéria sem zoom horizontal e sem instruções, usando os atalhos de matérias em alta.
Q2	Desempenho	Uma busca com filtro de matéria é submetida.	A lista é atualizada em até 2 s no P95, descontado o <i>cold start</i> da primeira chamada do dia.
Q3	Segurança	Um usuário autenticado chama <code>PATCH /api/sesoes/{id}/status</code> de uma sessão em que não é participante.	Resposta 403, nenhuma alteração de estado e o evento registrado na telemetria.
Q4	Confiabilidade	O serviço de e-mail fica indisponível durante o aceite de uma solicitação.	A sessão transiciona normalmente, a notificação in-app é gravada, a falha é registrada como aviso e a requisição responde 200.
Q5	Robustez	A API fica fora do ar enquanto a tela de busca carrega.	A interface exibe "Não foi possível carregar os mentores" com o motivo, em vez de tela em branco ou <i>spinner</i> infinito.
Q6	Corretude do domínio	Um mentor sem nenhuma avaliação aparece na busca.	O card mostra "Sem avaliações ainda", nunca 0.0, e o mentor é ordenado ao final da lista.
Q7	Manutenibilidade	Um novo desenvolvedor precisa alterar a regra de quem pode cancelar uma sessão.	A mudança acontece em um único componente (<code>ServicoDeSessao</code>), sem tocar no front-end nem nos handlers HTTP.
Q8	Custo	A plataforma passa um mês inteiro com uso esporádico.	O custo permanece dentro do crédito Azure for Students, com os recursos serverless escalando a zero nos períodos ociosos.

11. Riscos e Dívidas Técnicas

11.1 Riscos

RISCO	IMPACTO	MITIGAÇÃO
Latência de <code>centralus</code> para usuários no Brasil	Médio	O front-end é servido pela rede de borda, o que atenua o efeito nos arquivos estáticos. Migrar para <code>brazilsouth</code> assim que a policy da assinatura permitir.
<i>Cold start</i> na primeira requisição após ociosidade	Baixo	Flex Consumption reduz o tempo de partida. Se incomodar, avaliar instâncias sempre prontas — ao custo de sair do escalar-a-zero.
Esgotamento do crédito Azure for Students	Alto	Todos os recursos em SKU gratuita ou de consumo. Alerta de orçamento na assinatura e possibilidade de recriar o ambiente inteiro com <code>infra.sh</code> .
Integração com o Entra External ID ainda não validada	Alto	É o item que mais pode surpreender no cronograma. Fazer uma prova de conceito isolada do fluxo de login antes da sprint de identidade.
Dependência de um único ambiente, sem staging	Médio	Usar os ambientes de <i>preview</i> do Static Web Apps por pull request antes de publicar em produção.

11.2 Dívidas técnicas assumidas

DÍVIDA	POR QUE FOI ACEITA	COMO SERÁ PAGA
Dados mock em memória no lugar do banco	Permite entregar a integração front-end/Functions antes de o banco existir.	Substituir o módulo de dados pelos repositórios sobre PostgreSQL, mantendo os handlers e as regras.
Endpoints anônimos, sem autenticação	Os dados publicados são fictícios e somente de leitura.	Introduzir o <code>AuthMiddleware</code> e mudar o <code>authLevel</code> junto com a sprint de identidade.
Usuário fixo na listagem de sessões	Sem autenticação, não há de onde tirar o usuário corrente.	Derivar o usuário do token depois da integração com o Entra.
Publicação manual por CLI, sem pipeline	A conta usada não tinha permissão de criar workflows no repositório.	Adicionar GitHub Actions com <i>build</i> , publicação e ambiente de preview por pull request.
Ausência de testes automatizados	O escopo atual é leitura sobre dados fixos, verificada manualmente.	Testes de unidade sobre a máquina de estados e o cálculo de reputação, que são a parte com regra de verdade.
Infraestrutura em script imperativo	<code>infra.sh</code> resolveu a necessidade imediata com o ferramental já instalado.	Migrar para Bicep ou Terraform quando o número de recursos justificar.

12. Glossário

TERMO	DEFINIÇÃO
Calouro / mentorado	Aluno que busca ajuda. No modelo de dados, um <code>Usuario</code> com <code>ehMentorado = true</code> .
Veterano / mentor	Aluno que oferece mentoria. Um <code>Usuario</code> com <code>ehMentor = true</code> . Os dois papéis podem coexistir na mesma pessoa.
Área de mentoria	Par curso + matéria em que um mentor se declara apto a ajudar. Sem ao menos uma, o mentor não aparece na busca.
Disponibilidade	Bloco recorrente de dia da semana e faixa de horário em que o mentor aceita ser agendado.
Sessão	Encontro de mentoria entre um mentor e um mentorado, com estado governado pela máquina da seção 6.2.
Reputação	Média aritmética das notas recebidas por um mentor, com uma casa decimal, acompanhada do total de avaliações. Mentor sem avaliação não tem reputação — e não exibe 0.0.
arc42	Template livre para documentação de arquitetura de software, organizado em doze seções.
C4 Model	Notação de diagramas de arquitetura em quatro níveis de zoom: Contexto, Contêineres, Componentes e Código.
DER	Diagrama de Entidades e Relacionamentos: representa as entidades persistidas, seus atributos e a cardinalidade entre elas.
Cold start	Latência adicional da primeira requisição a uma função serverless que estava sem instância ativa.
Best effort	Operação que se tenta executar, mas cuja falha não interrompe nem invalida o fluxo principal.
Managed Identity	Identidade gerenciada pela Azure e atribuída a um recurso, permitindo autenticação em outros serviços sem credencial no código.